Support Vector Machine

Understanding Support Vector Machines

From Intuition to Mathematical Formulation

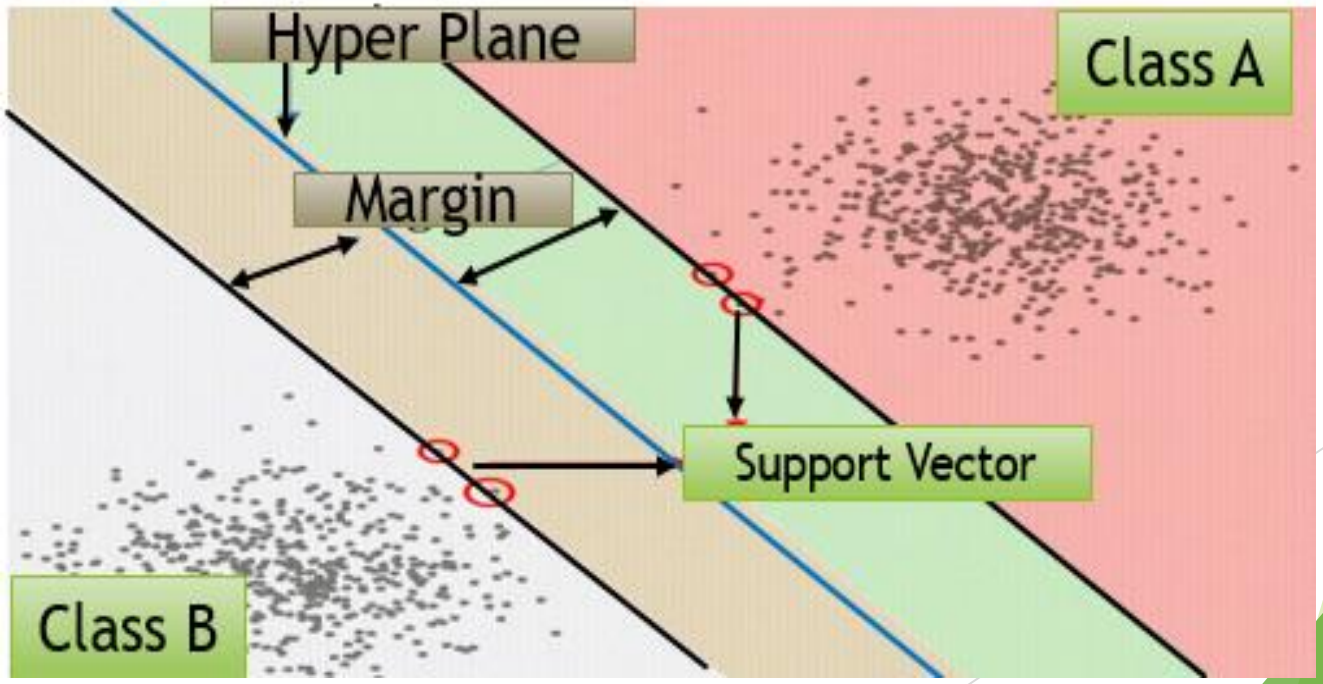
In 1990s while neural networks were struggling due to computational limits, **SVM** became a very powerful alternative.

It can handle both **regression and classification problems** for linear and nonlinear data.

However it is most preferred for **classification problems**.

What is SVM?

- **Support Vector Machine**, or SVM, is a supervised learning algorithm used for classification and regression tasks
- Linear SVM (LinearSVC) → for linearly separable data
- Kernel SVM (SVC) → for nonlinear data
- Its primary goal is to find the **optimal hyperplane** that best separates data points belonging to different classes.



So if we are about to study for SVM.

We need to understand,

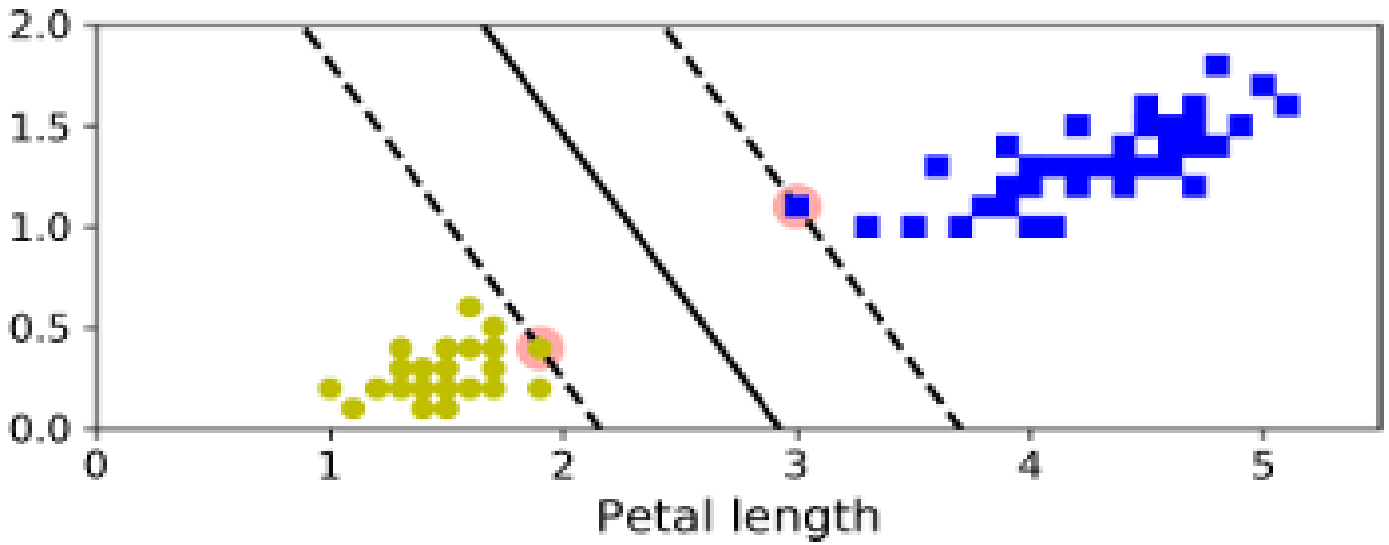
What is **hyper plane**?

What is **support vector**?

What is **margin**?

Hyperplane:

- A hyperplane is a decision boundary that separates data points into different classes.
- In a 2D space, it’s a line; in a 3D space, it’s a plane; and in higher dimensions, it’s a hyperplane.
- A hyperplane is equidistant from the closest data points of both classes.
- These closest points are called support vectors.



- Solid line in the above plot represents the hyperplane of an SVM classifier.
- This line not only separates the two classes but also stays as far away from the closest training instances as possible.

- You can think of an SVM classifier as fitting the widest possible street (represented by the parallel dashed lines) between the classes. This is called large margin classification.
- Here adding more training instances “off the street” will not affect the decision boundary at all.

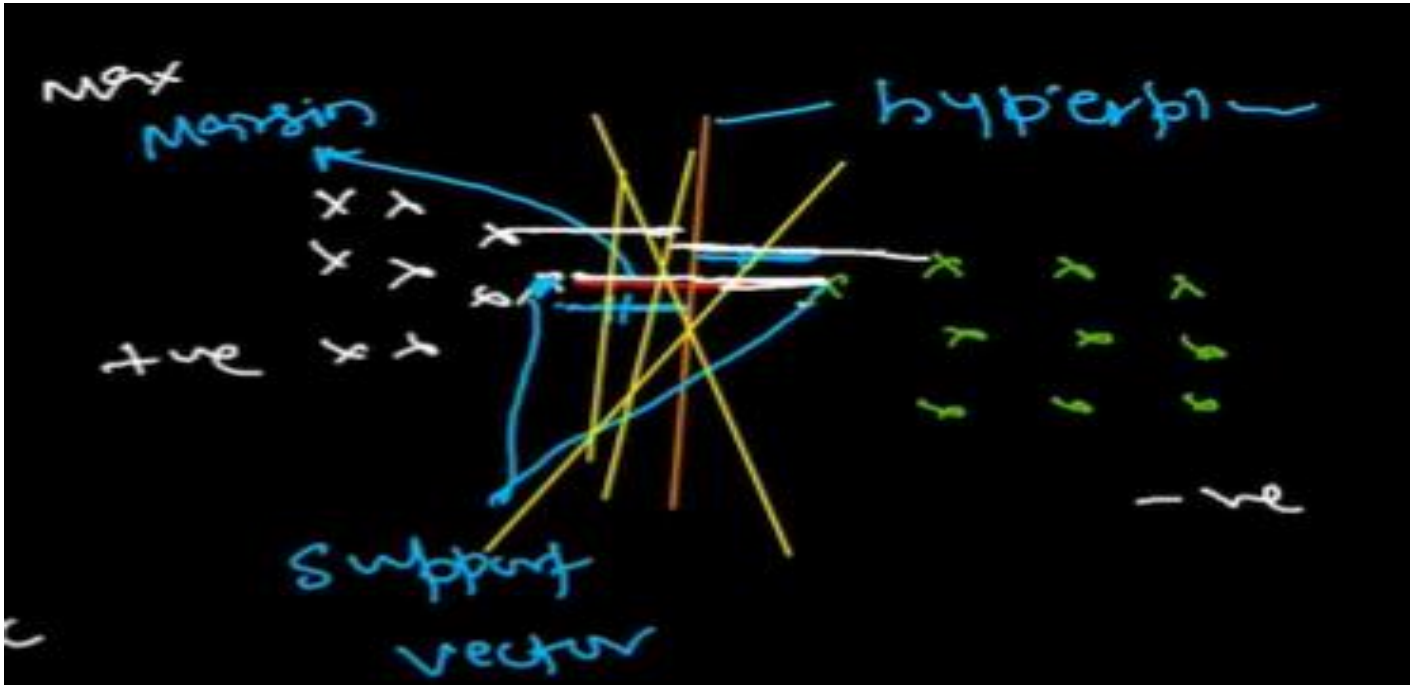
Support Vectors:

- Support vectors are the data points closest to the hyperplane.
- These points determine the position and orientation of the hyperplane.

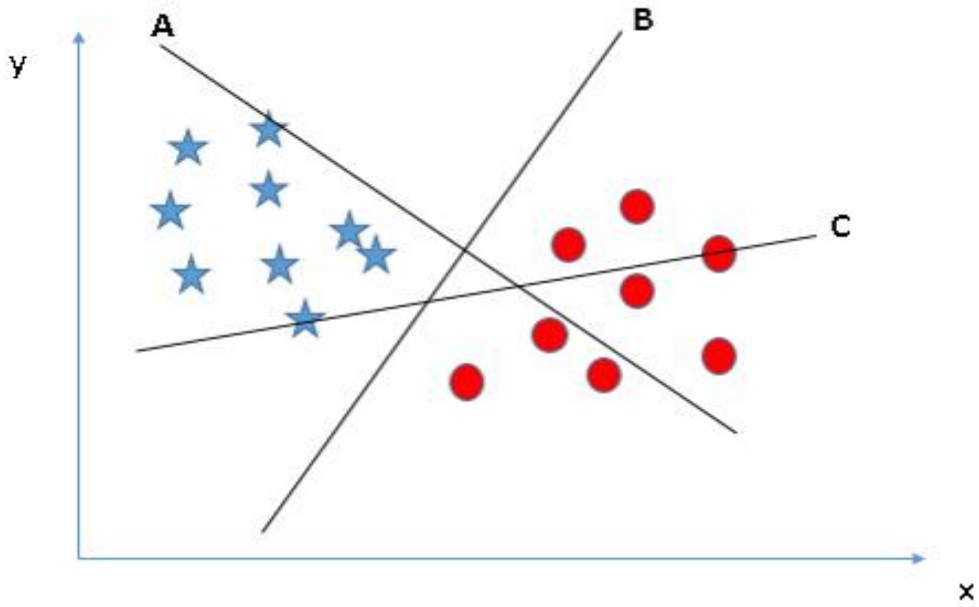
Margin:

- The margin is the distance between the hyperplane and the nearest data points from each class.
  - **Large Margin:** Preferred as it implies better generalization. It is a good margin.
  - **Small margin** may lead to poor generalization.
  - Therefore, a larger margin is preferred.**Hard Margin:** Primal solution with linearly separable data.
  - **Soft Margin:** for non linearly separable data, allowing some misclassification.

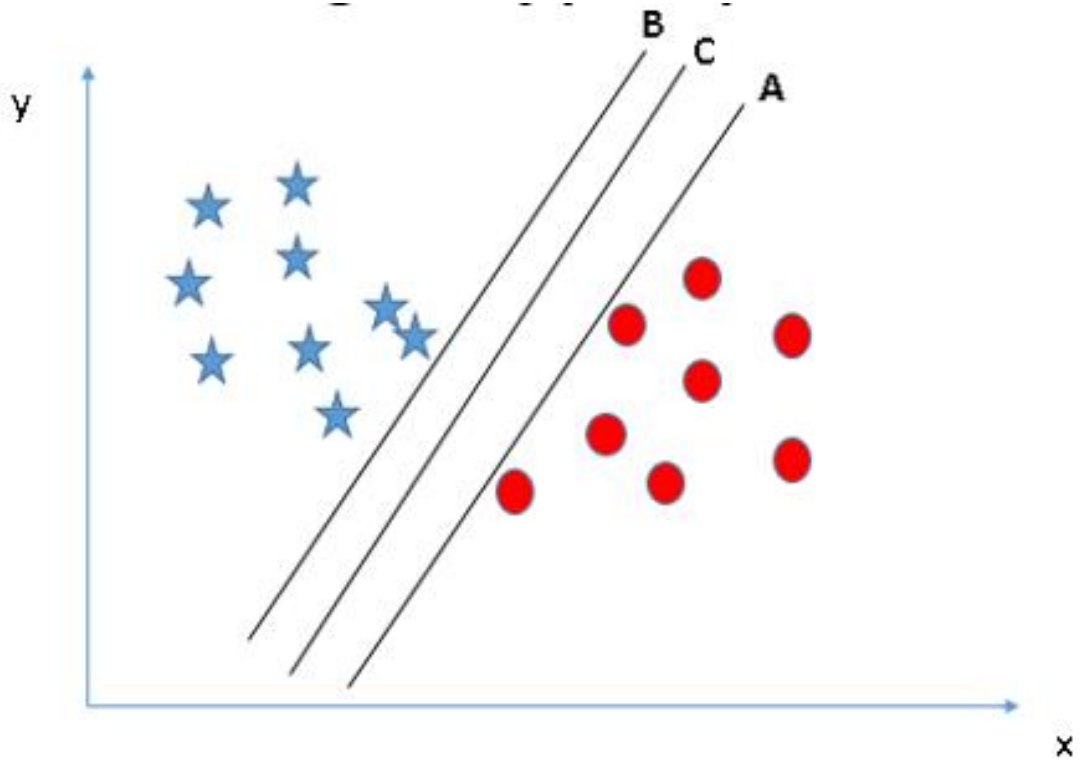
- **If two classes can be separated by a straight line (or hyperplane), the data is linearly separable.**
- **Similar to logistic regression, many lines or planes can separate the classes.**
- **However, SVM selects the hyperplane that maximizes the margin.**
- **But the plane that separate positive and negative point as wide as possible is the best line.**


max
Margin
hyperpl
+ve
-ve
Support
vector

Now tell me which one is the best hyperplane here?

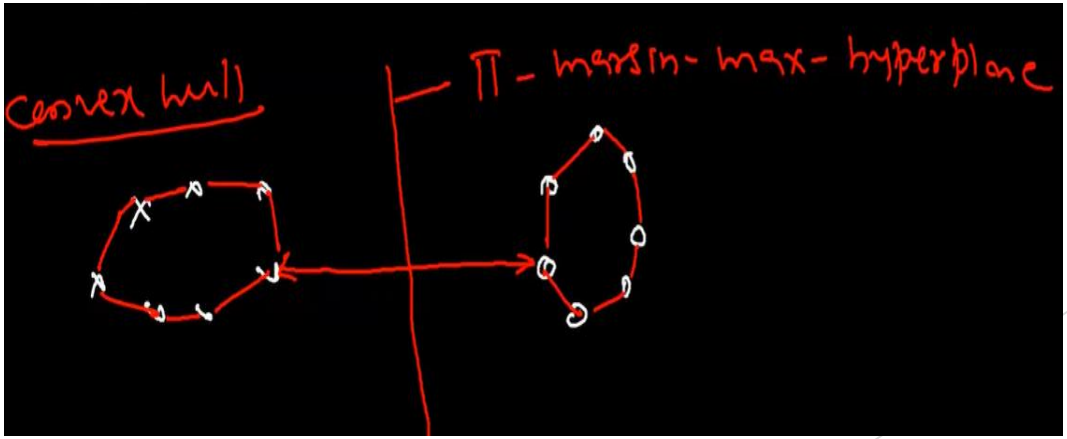


What about this?



Convex Hull

- The **convex hull** is the smallest convex polygon that contains all the points of that class.
- If data is linearly separable: The two convex hulls **do not intersect**.
- Find the **shortest line segment connecting the two hulls**.
- This line segment connects two boundary points. Those boundary points are **support vectors**
- **T**ake that shortest line segment, Draw a plane that **bisects it perpendicularly**
- That plane is: Maximum Margin Hyperplane



What is margin width(λ) ?

In hard-margin SVM we define:

$$w^T x + b = 1$$

$$w^T x + b = -1$$

These are two parallel planes.

The optimal separating hyperplane is:

$$w^T x + b = 0$$

So margin width is:

$$\text{Margin} = \frac{2}{||w||}$$

That is the **distance between those two planes**.

SVM actually defines three parallel planes.

1️⃣ Decision Boundary (Middle Plane)

$$w^T x + b = 0$$

This is the classifier.

2️⃣ Positive Margin Plane

$$w^T x + b = +1$$

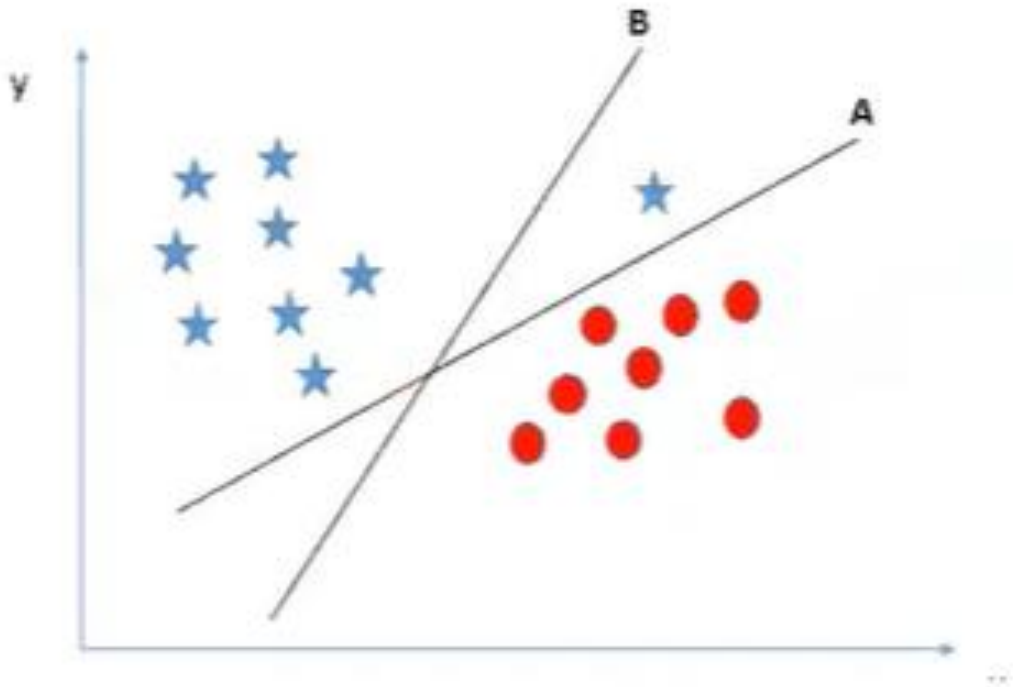
Positive support vectors lie here.

3️⃣ Negative Margin Plane

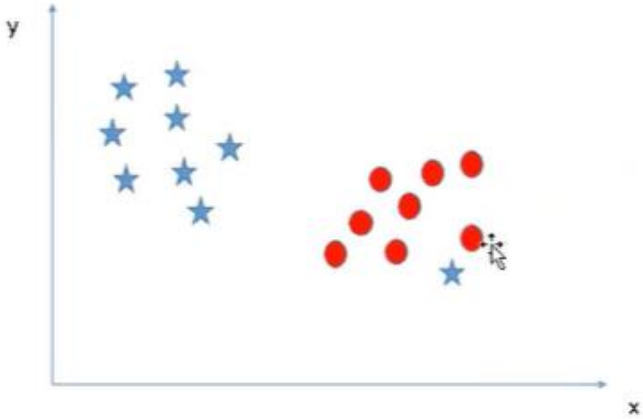
$$w^T x + b = -1$$

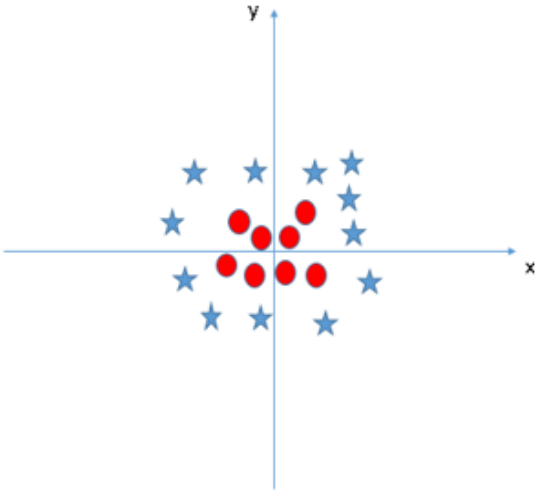
Negative support vectors lie here.

Which one is better?



But the challenge starts here

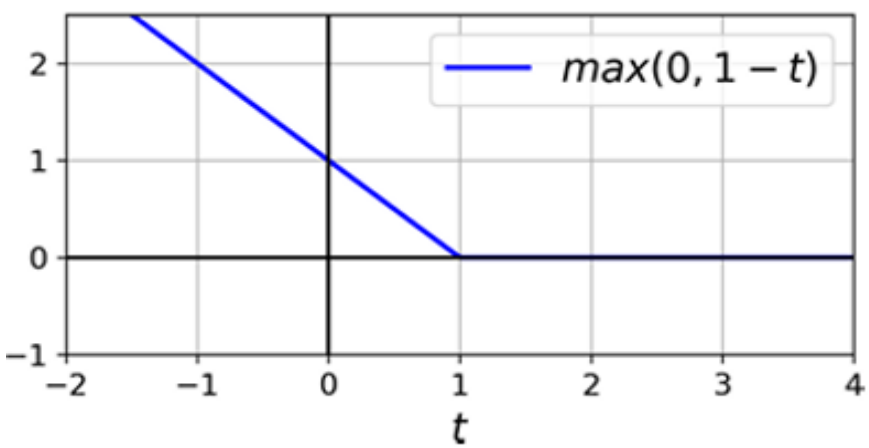




Hinge Loss:

- It is a loss function used in Support Vector Machines (SVMs) for binary classification tasks. It helps in maximizing the margin between classes, ensuring robust separation of data points.
- Hinge loss penalizes points inside the margin or misclassified, but ignores those safely outside.
- **L(t)=max(0,1−t)**

$$(w^*, b^*) = \arg\max_{w,b} \frac{2}{||w||}$$



What is t ?

- **$t = y \cdot f(x)$**
- It is called the **margin value**.
- It measures how correct the prediction is, not what the class is.
- Where y is the **true class label**.
- **For SVM**: $y \in \{-1,+1\}$
- f(x)=w.x+b-->.It is the **model score**, not the class.
- **Predicted class is:**
- $\hat{y} = \text{sign}(f(x))$
- So: If $f(x) > 0 \rightarrow +1$,
- If $f(x) < 0 \rightarrow -1$

- So hinge loss depends on: yf(x) which measures **how correct the prediction is**.

Value of t	**Meaning**	**Loss**
(t > 1)	Correct	0 and outside margin
(0 < t < 1)	Correct	>0 but inside margin
(t < 0)	Misclassified	large

Loss minimization ⇒ LOG REG ⇒ Logistic Loss + Regularizer

Linear REG ⇒ Squared Loss + Regularizer

SVM ⇒ Hinge loss + Regularizer

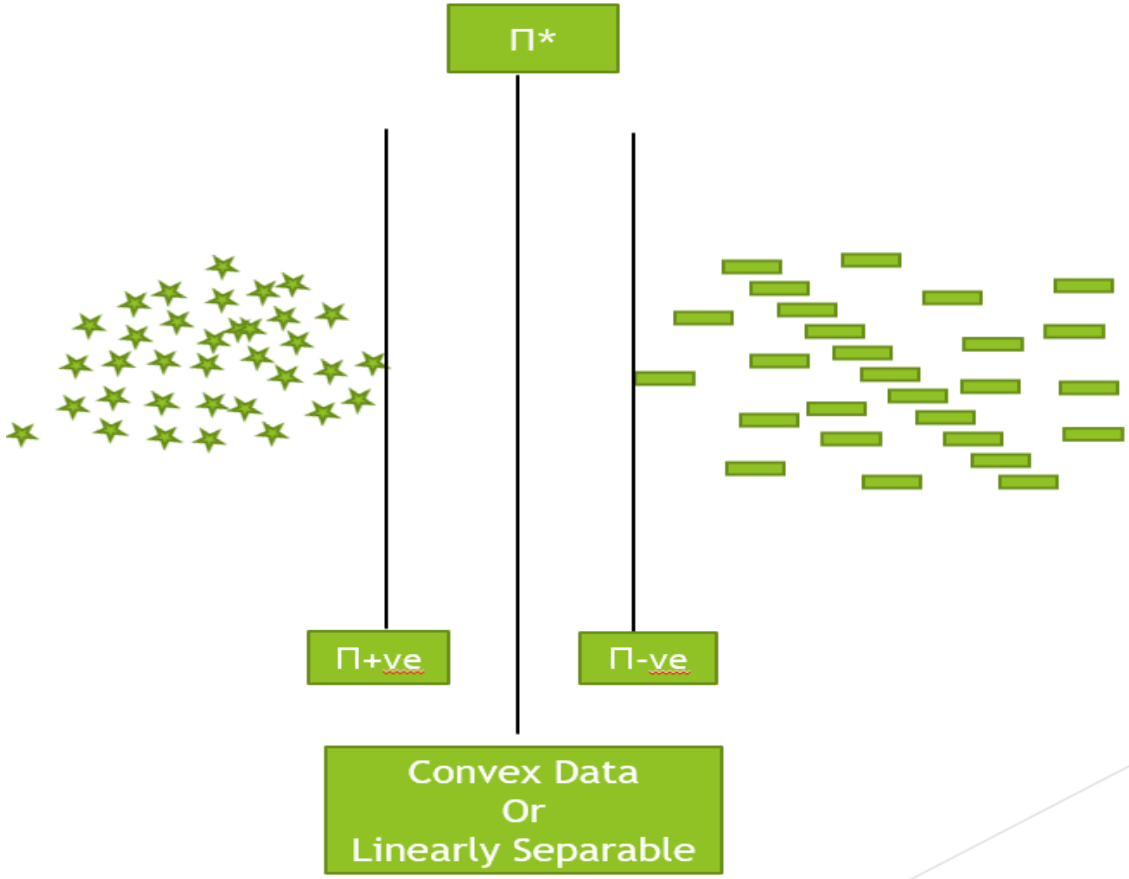
How SVM Works?

- **Finding the Hyperplane:**
  - Maximizes the margin between different classes.
  - Uses support vectors to determine the decision boundary.
- **Decision Boundary:**
  - Separates positive and negative classes as widely as possible.

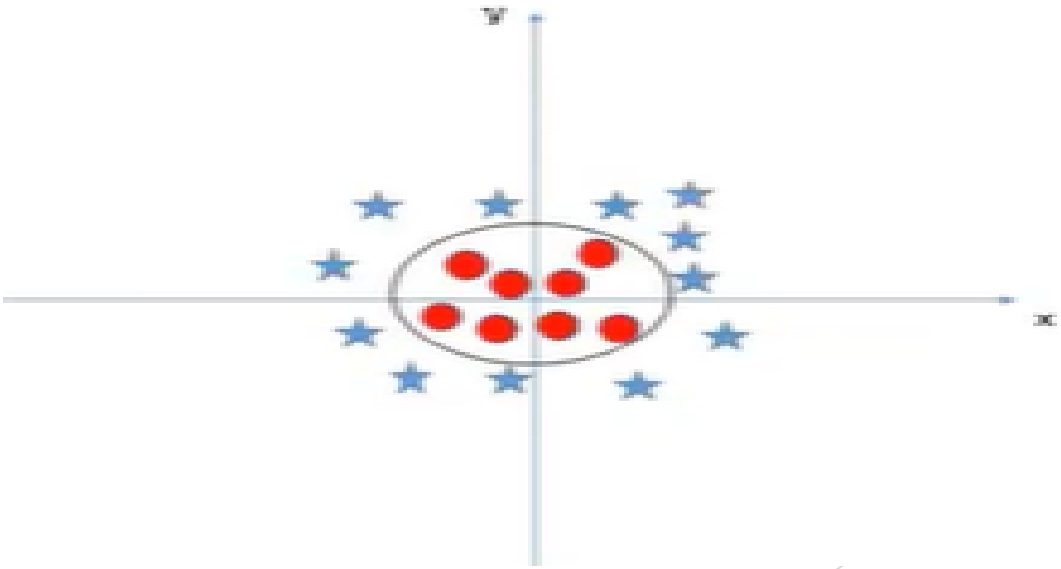
Linearly separable data

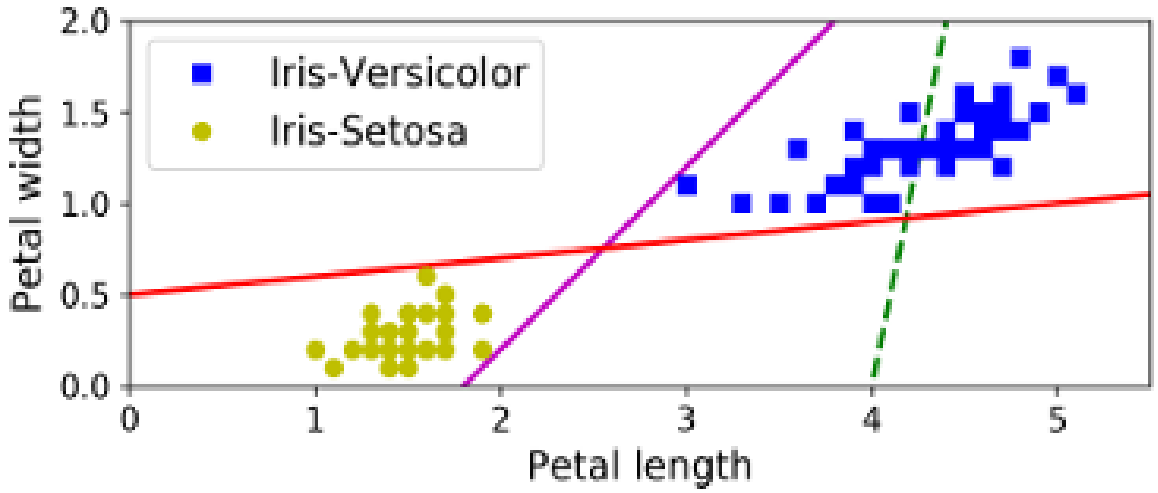
$$K(x, y) = x^T y$$

- In case of **linearly separable data** we can draw thousands of line between two classes and looking for a line which at equidistant from two classes which is called hyperplane.



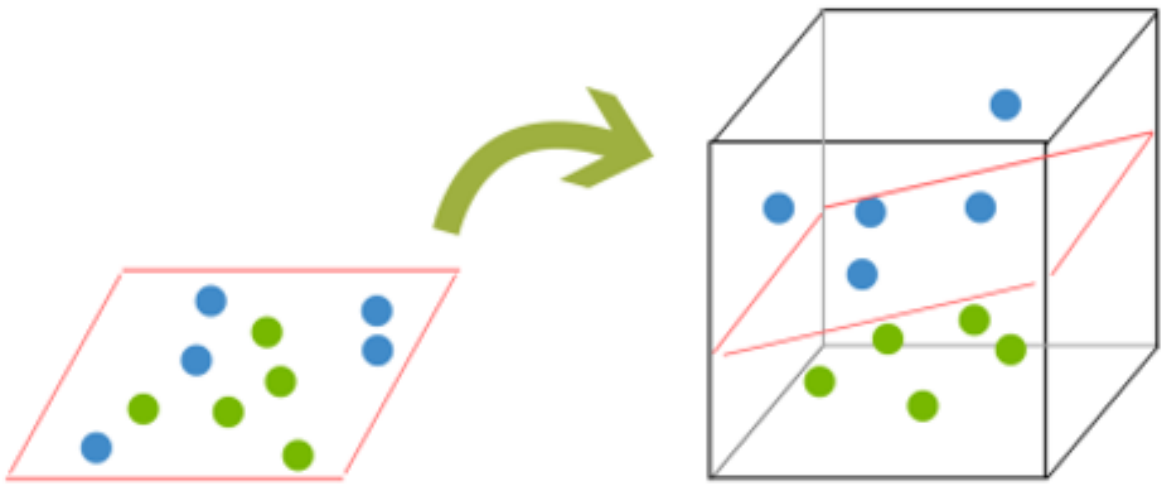
- If data is not linearly separable in the original feature space,
- we use kernel methods to make it linearly separable in a higher-dimensional space.
- Kernel lifts data so straight line works





- The plot shows the decision boundaries of three possible linear classifiers.
- The model with decision boundary by the **dashed line** can not separate the classes properly.
- The other two models work perfectly, but their decision boundaries come so close to the instances that makes them difficult to perform well on new instances.

2d to 3d



Step 1: Original 2D Data (Not Linearly Separable)

Point	x_1	x_2	Class
A	1	0	+1
B	0	1	+1
C	-1	0	+1
D	0	-1	+1
E	0	0	-1

No straight line can separate center from all outer points.

Step 2:Define Feature Transformation

We define:

$$\phi(x) = (x_1, x_2, x_1^2 + x_2^2)$$

So we are mapping:

2D → 3D

Step 3: Apply Transformation

Point A: (1,0)

$$\phi(A) = (1, 0, 1^2 + 0^2) = (1, 0, 1)$$

Point B: (0,1)

$$\phi(B) = (0, 1, 1)$$

Point C: (-1,0)

$$\phi(C) = (-1, 0, 1)$$

Point D: (0,-1)

$$\phi(D) = (0, -1, 1)$$

Point E: (0,0)

$$\phi(E) = (0, 0, 0)$$

Step 4: Look at New Coordinates

Outer points → z = 1

Center point → z = 0

So in 3D:

- Four positive points lie on plane z = 1
- Negative point lies at z = 0

Now we can separate with a flat plane:

$$z = 0.5$$

Which means:

If $x_1^2 + x_2^2 > 0.5$ → +1

Else → -1

That is circular boundary in 2D.

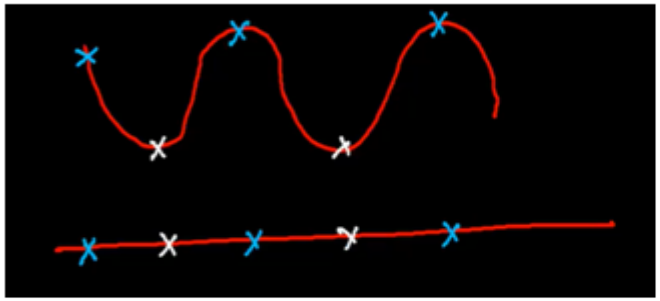
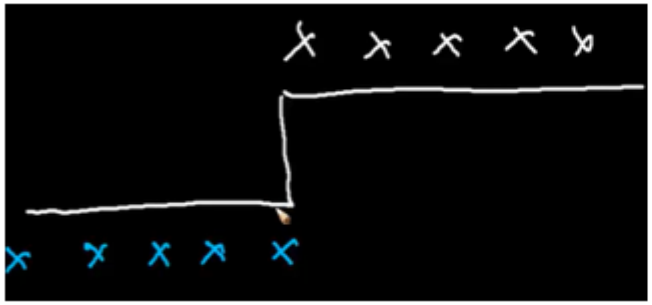
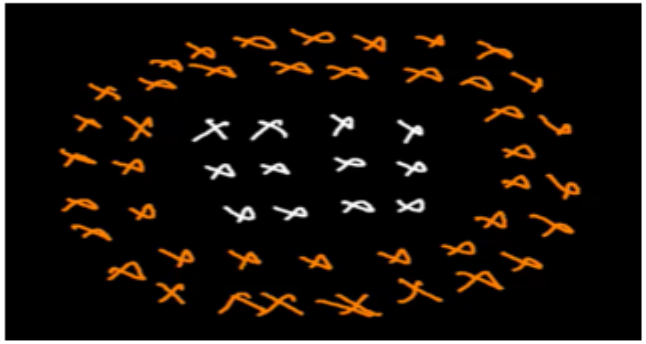
So nonlinear in 2D becomes linear in 3D.

- Feature mapping lifts data into higher dimension where a simple linear separator becomes possible.

Kernel

- In SVM, we always try to find a **linear hyperplane** that separates classes.
- But sometimes data is **not linearly separable in the original feature space**.
- Then SVM uses a technique called the **Kernel Trick**.
- A kernel function:
- Takes low-dimensional input space
- Implicitly maps it to a higher-dimensional space
- Makes non-separable data linearly separable
- Then SVM finds a linear hyperplane in that higher space.

Common kernel Tricks

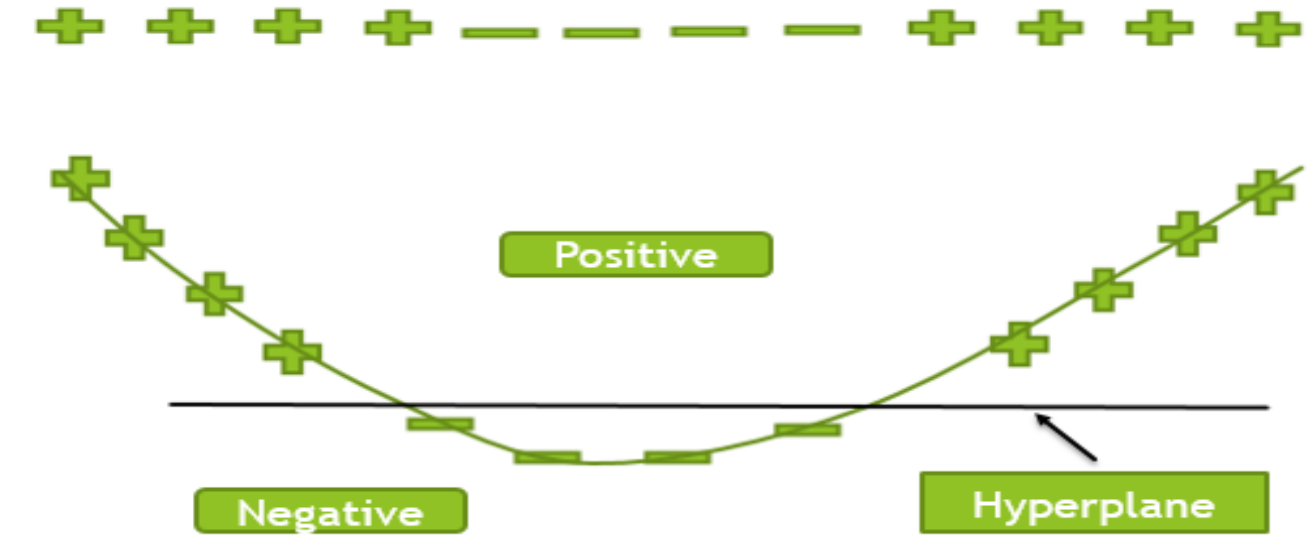
Shape of DATA	Kernel Trick
	Polynomial
	Sigmoid
	rbf

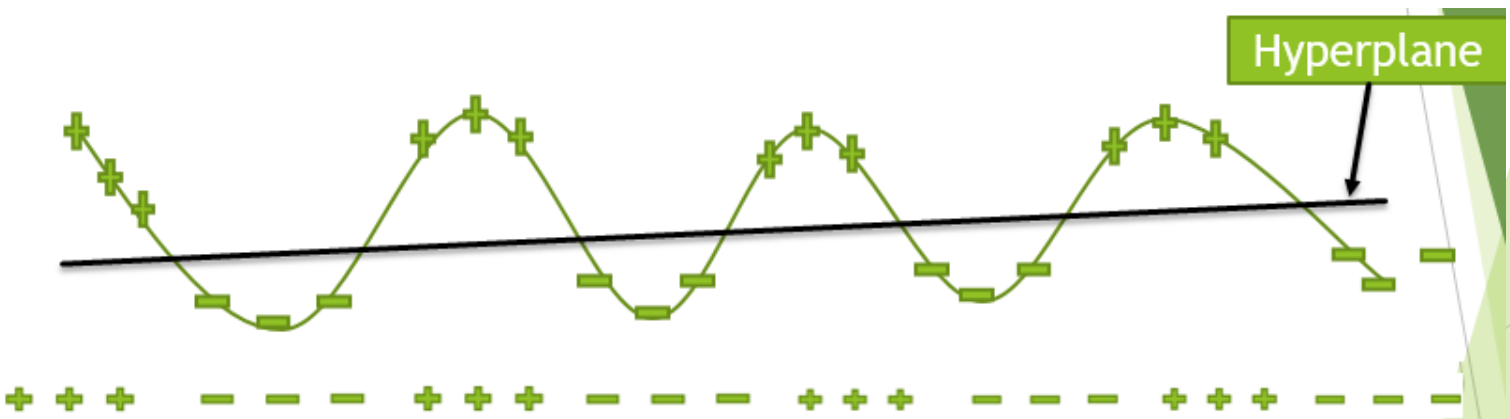
- But in case of non **linearly separable data** it is not simple to draw hyperplane and polynomial kernel come into picture.

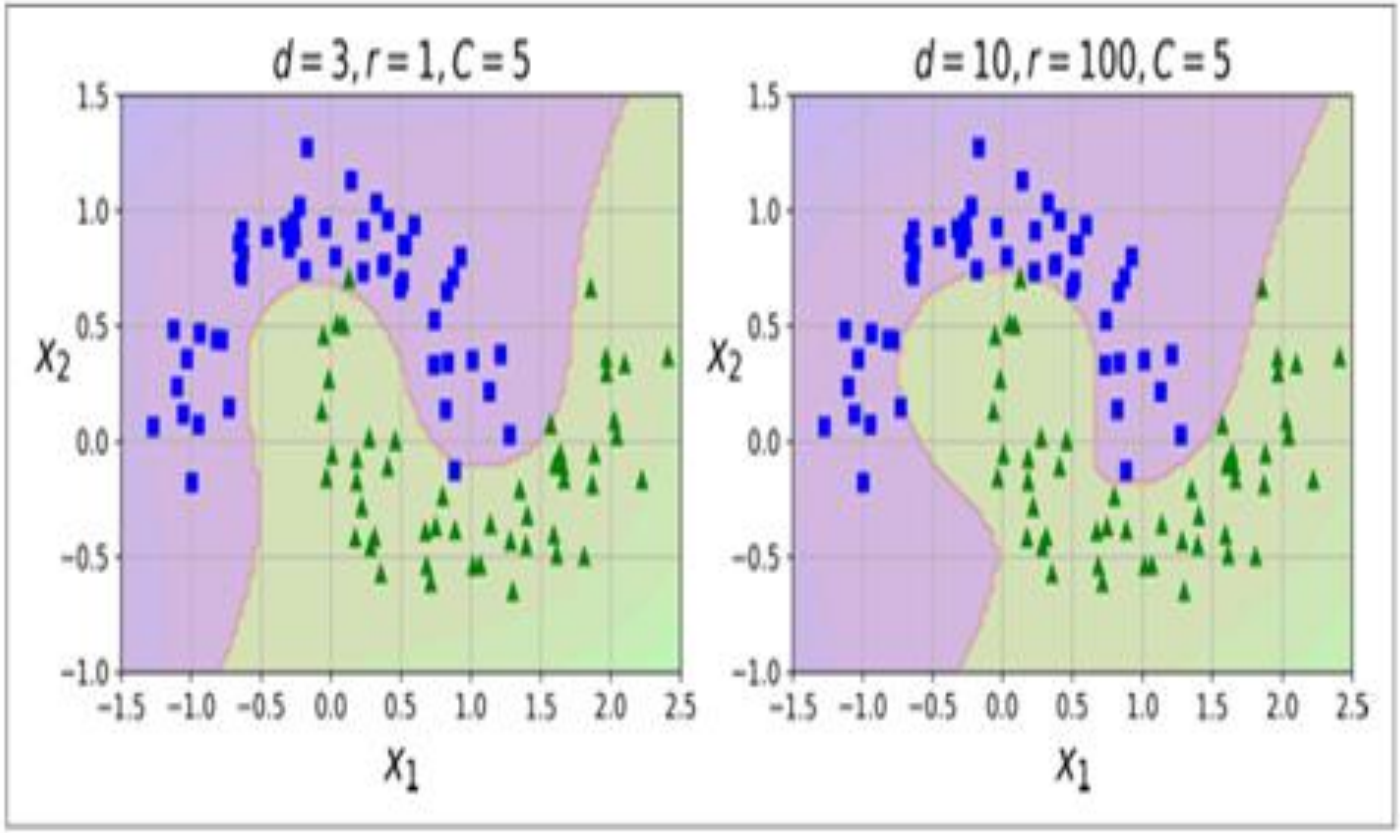
Kernel = Polynomial

- Polynomial kernel creates polynomial decision boundary, means where we are increasing the magnitude of each point polynomially.

$$K(x, y) = (\gamma x^T y + r)^d$$


Positive
Negative
Hyperplane


Hyperplane


$d = 3, r = 1, C = 5$
$d = 10, r = 100, C = 5$
X_1
X_2
1.5
1.0
0.5
0.0
−0.5
−1.0
−1.5
−1.0
−0.5
0.0
0.5
1.0
1.5
2.0
2.5

Polynomial kernel expands

For d = 2:

$$(x_1 + x_2)^2 = x_1^2 + 2x_1x_2 + x_2^2$$

For d = 10:

It creates very high-order interaction terms.

That's why boundary becomes very complex.

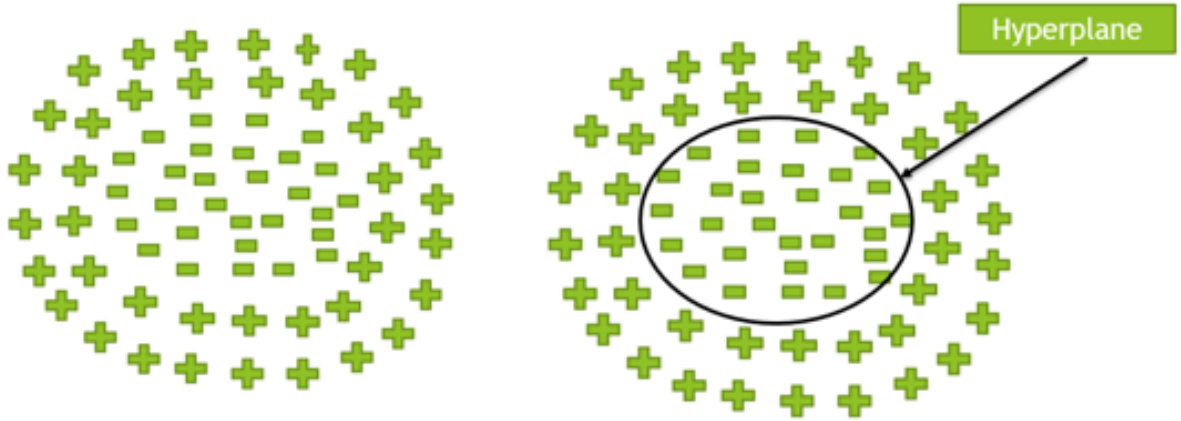
Parameter	Meaning	Effect
d	Polynomial degree	Controls boundary complexity
r	Bias term (coef0)	Controls influence of lower-order terms
C	Regularization	Controls overfitting vs margin

Kernel = RBF
Radial Basis Function

$$K(x, y) = \exp(-\gamma ||x - y||^2)$$

- It measures similarity between two points.
- $\|x - y\|^2$ = squared Euclidean distance between x and y
- If two points are: Very close → distance small
- Far apart → distance large
- Then: $\exp(-\gamma \cdot \text{distance})$
- If distance small: K(x, y)≈1 → Very similar
- If distance large: K(x,y)≈0 → very different

- **Why “Radial”?**
- Because similarity depends only on distance.
- It forms circular (radial) influence around each point.
- Instead of drawing straight line: RBF allows curved boundary.
- What Is γ (Gamma)?
- Controls how far influence of a point spreads.



Class is predicted using:

$$f(x) = \sum \alpha_i y_i K(x_i, x) + b$$

Then:

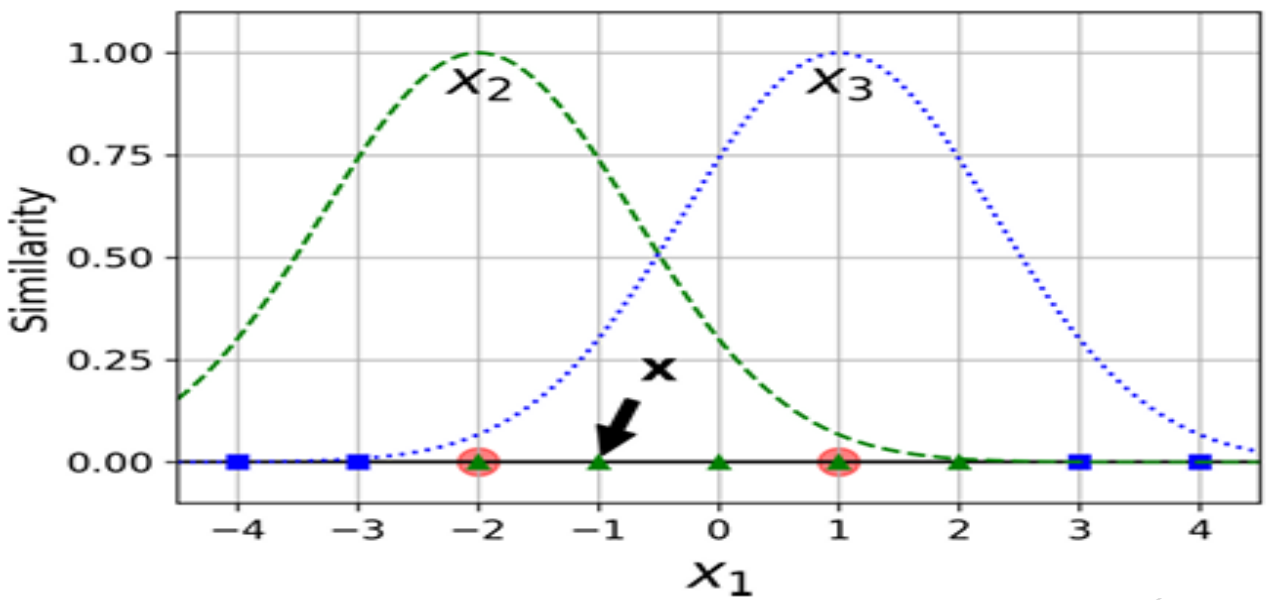
$$\hat{y} = \text{sign}(f(x))$$

So:

- $K(x_i, x)$ → similarity
- $f(x)$ → score
- sign(f(x)) → class

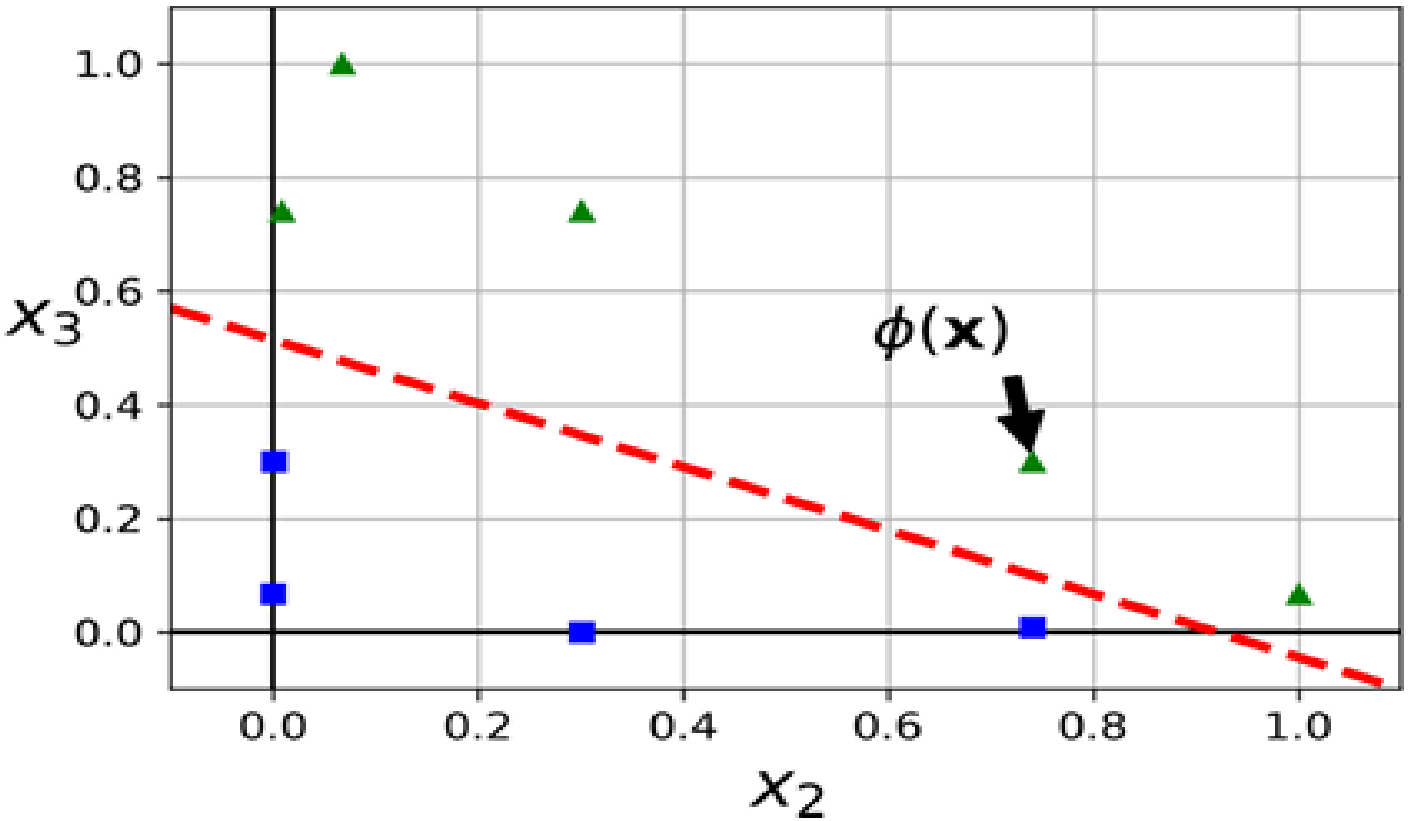
Example:

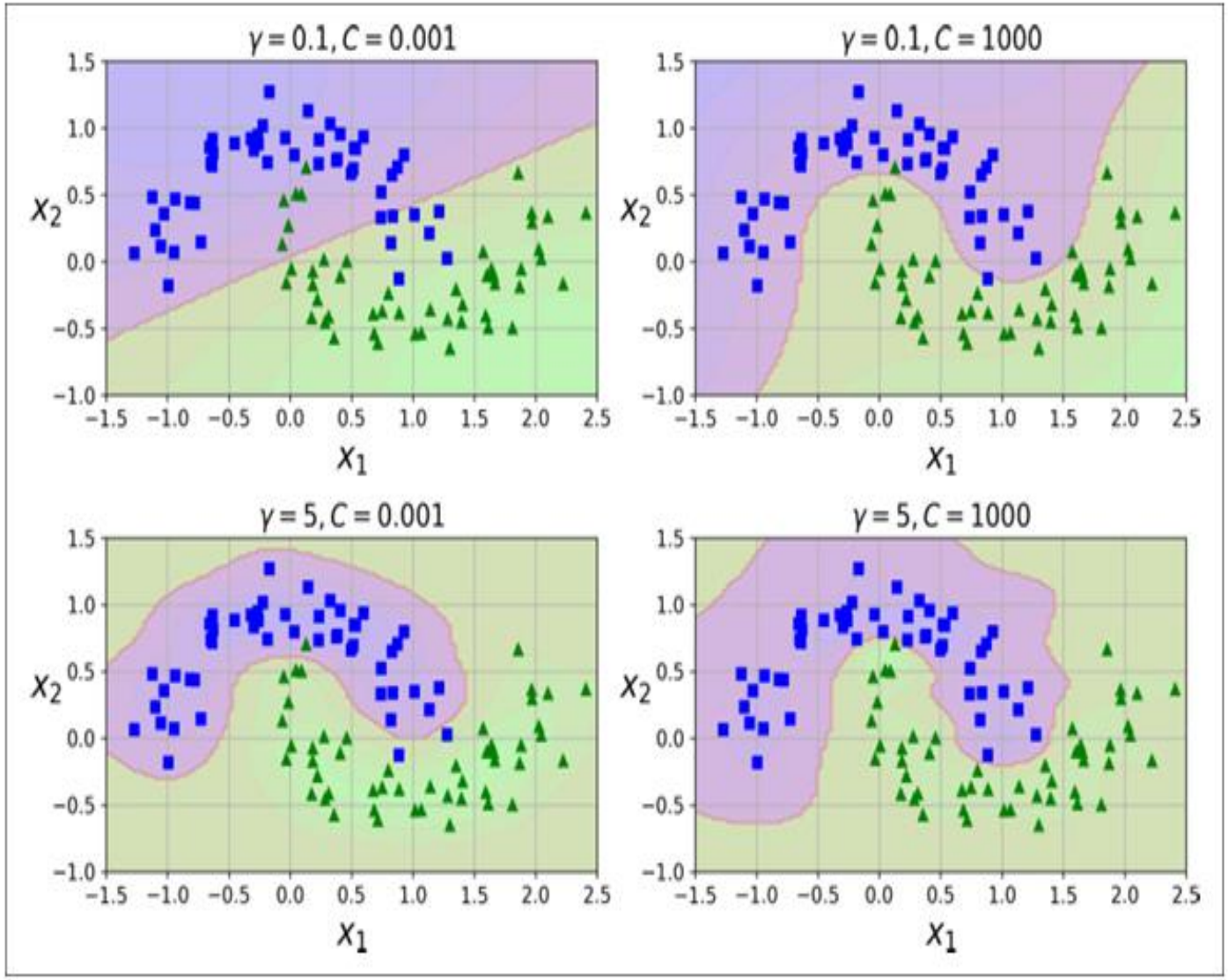
- let's take the one-dimensional dataset ,add two landmarks to it at x1 = –2 and x1 = 1 (see the plot).
- Next, let's define the similarity function to be the Gaussian Radial Basis Function (RBF) with γ = 0.3 .



- It is a bell-shaped function varying from 0 (very far away from the landmark) to 1 (at the landmark).
- To compute the new features. For example, let's look at the instance
- $x1 = -1$:
- it is located at a distance of 1 from the first landmark, and 2 from the second landmark.
- Therefore its new features are :
- $x_2 = \exp(-0.3 \times 1^2) \approx 0.74$
- $x_3 = \exp(-0.3 \times 2^2) \approx 0.30$.

- The plot shows the trans formed dataset (dropping the original features).
- Now linearly separable.



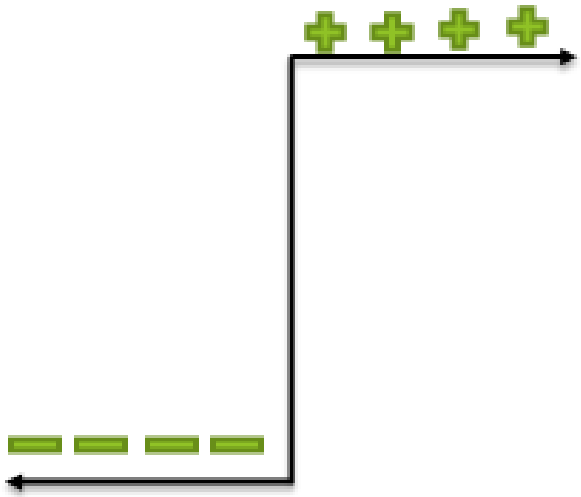

γ = 0.1, C = 0.001
γ = 0.1, C = 1000
γ = 5, C = 0.001
γ = 5, C = 1000
x1
x2

- The plots show models trained with different values of hyper parameters gamma (γ) and C.
- The plots show that increasing gamma makes the bell-shape curve narrower, and as a result each instance's range of influence is smaller.
- Conversely, a small gamma value makes the bell-shaped curve wider, so instances have a larger range of influence.
- So γ acts like a regularization hyperparameter: if your model is overfitting, you should reduce it, and if it is under fitting, you should increase it

- This balance is crucial because it determines whether the SVM will fit the training data too closely (overfitting) or be too general (underfitting).
- **High gamma:** Model is sensitive to training data, potentially overfitting.
- **Low gamma:** Model is smoother and more generalized, potentially underfitting.

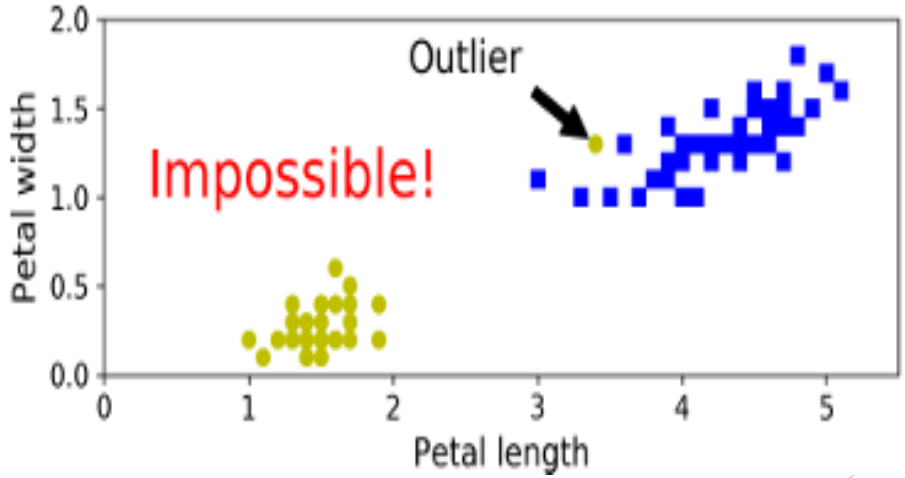
Kernel = Sigmoid

$$K(x, y) = \tanh(\gamma x^T y + r)$$

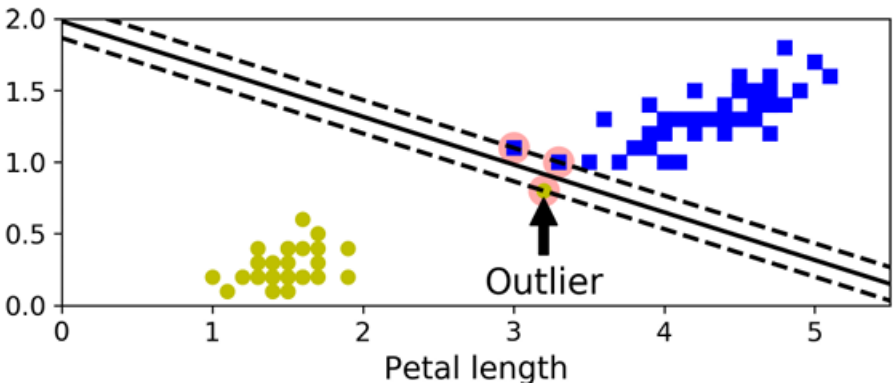


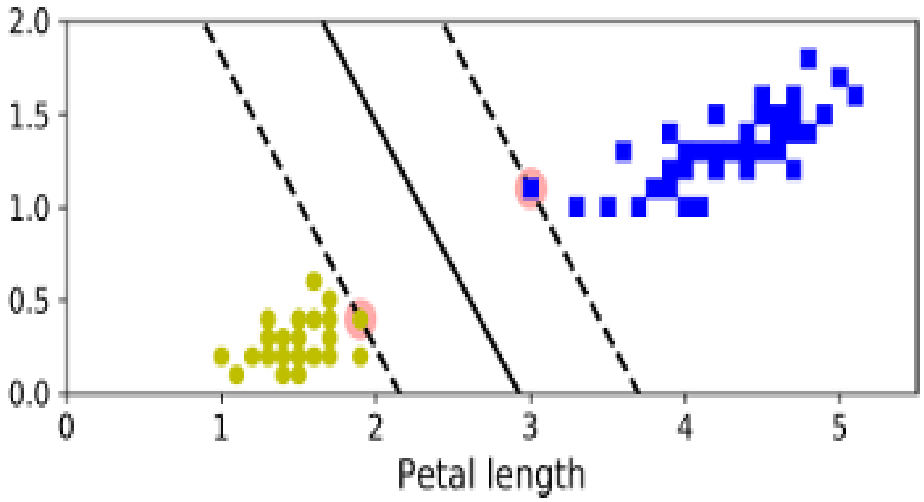
Hard Margin limitation

- If we strictly impose that all instances be ‘off the street’ and on the right side, this is called **hard margin** classification.
- There are two main issues with hard margin classification.
  - First, it only works if the data is linearly separable, and
  - Second it is quite sensitive to outliers.
  - Just one additional outlier: it is impossible to find a hard margin.



- While the decision boundary ends up very different without the outlier.





- To avoid these issues it is preferable to use a more flexible model.
- The objective is to find a good balance between keeping **the street as large as possible** and **limiting the margin violations** (i.e., instances that end up in the middle of the street or even on the wrong side).
- This is called **soft margin classification.**
- This balance can be controlled using the C hyper parameter:

Hard margin linear SVM classifier objective

$$\underset{\mathbf{w}, b}{\text{minimize}} \quad \frac{1}{2}\mathbf{w}^T\mathbf{w}$$

$$\text{subject to} \quad t^{(i)}\left(\mathbf{w}^T\mathbf{x}^{(i)} + b\right) \geq 1 \quad \text{for } i = 1, 2, \cdots, m$$

Soft margin linear SVM classifier

- To get the soft margin objective, we need to introduce a slack variable $\zeta(i) \geq 0$ for each instance:
- $\zeta(i)$ measures how much the ith instance is allowed to violate the margin.
- We now have two conflicting objectives:
  - making the slack variables as small as possible to reduce the margin violations,
  - and making $\frac{1}{2}\mathbf{w}^T\mathbf{w}$ as small as possible to increase the margin.
  - This is where the C hyper parameter comes in:
    - it allows us to define the tradeoff between these two objectives.

Soft Margin SVM objective function.

$$\min \frac{1}{2}||w||^2 + C\sum_{i=1}^{n}\xi_i$$

- $\frac{1}{2}||w||^2$ This term controls the **margin size**.
- Remember:
- Margin = 2 / ||w||
- So: Minimizing ||w||2 → Maximizes margin.
- This part says: Keep the boundary as flat and simple as possible.

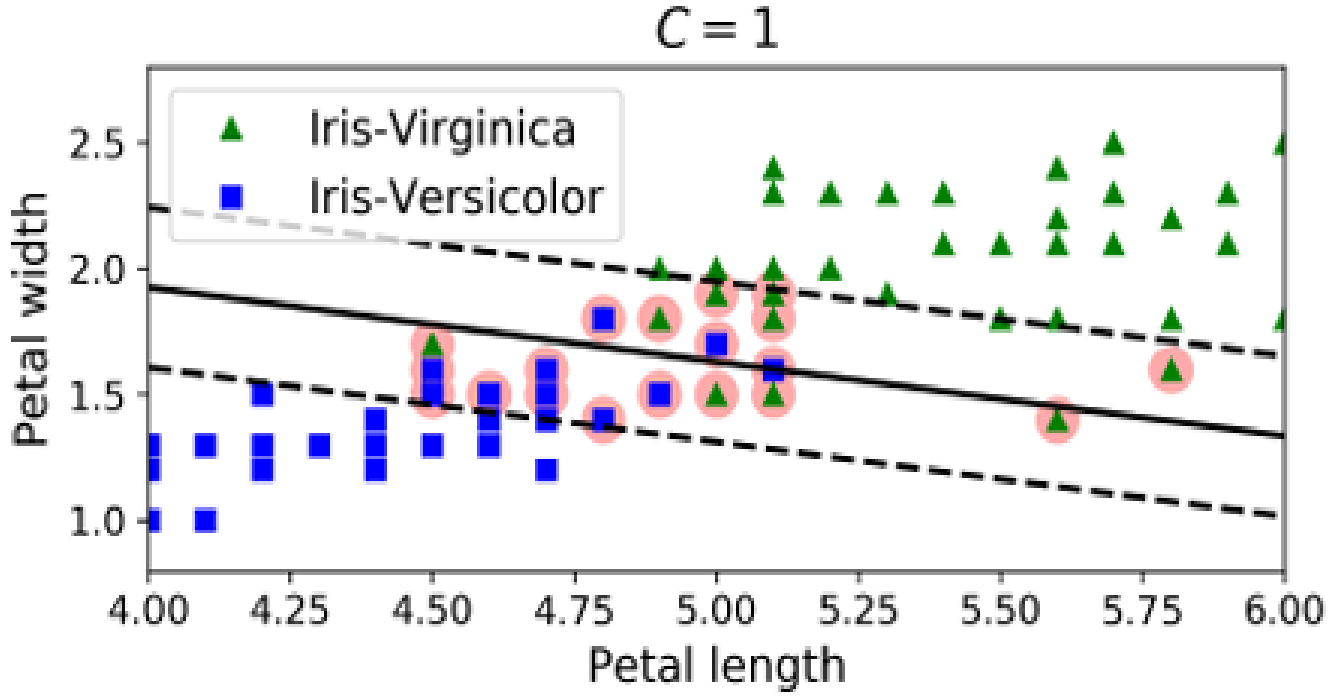
- $\sum \xi_i$:These are **slack variables.**
- They measure how much each point violates the margin.
- If:
  - $\xi_i = 0 \rightarrow$ correctly classified outside margin
  - $0 < \xi_i < 1 \rightarrow$ inside margin
  - $\xi_i > 1 \rightarrow$ misclassified

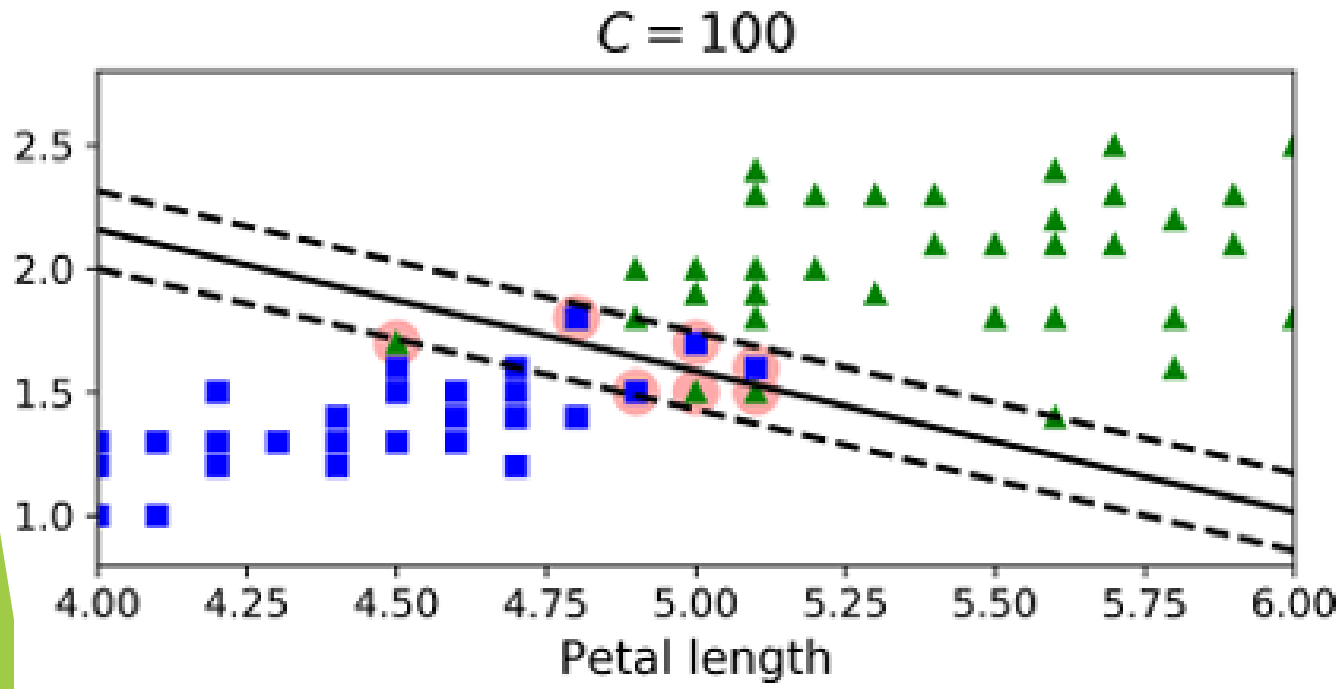
So this term counts total violation.

C is the regularization parameter

- It controls trade-off:
- Small C → ignore violations → wider margin

  Large C → punish violations → narrower margin
- So:
- C balances: Margin size vs Misclassification penalty


$C = 1$
Iris-Virginica
Iris-Versicolor
Petal width
2.5
2.0
1.5
1.0
4.00
4.25
4.50
4.75
5.00
5.25
5.50
5.75
6.00
Petal length


$C = 100$
2.5
2.0
1.5
1.0
4.00
4.25
4.50
4.75
5.00
5.25
5.50
5.75
6.00
Petal length

Geometric Interpretation

Without slack:

All points must satisfy:

$$y_i(w^T x_i + b) \geq 1$$

With slack:

We allow:

$$y_i(w^T x_i + b) \geq 1 - \xi_i$$

So margin can be violated.

Soft-margin SVM minimizes model complexity while penalizing classification errors.

Term	Meaning
$\|w\|^2$	Margin control
ξ_i	Violation amount
C	Trade-off parameter

Primal Form of SVM

- Many hyperplanes can separate the positive and negative.
- SVM chooses the hyperplane with the maximum margin.
- Also called decision boundary.

For soft-margin SVM:

$$\min_{w,b,\xi} \quad \frac{1}{2}||w||^2 + C\sum \xi_i$$

Subject to:

$$y_i(w^T x_i + b) \geq 1 - \xi_i$$

$$\xi_i \geq 0$$

This is called the **Primal Problem**.

Here we directly optimize:

- w
- b
- ξ

Why Dual Form?

- We introduce the **dual form** because it allows us to use the **kernel trick**, which is difficult to apply directly in the primal formulation.
- In the **dual formulation**, the optimization depends only on **dot products between data points**.
- These dot products can be replaced by a **kernel function**.

- The primal SVM works directly with the feature vectors, but the dual SVM depends only on dot products between training samples.
- This allows us to replace the dot product with a kernel function and implicitly map data into a higher dimensional space.
- Dual Form converts constrained problem into α optimization.
- Decision boundary depends only on training samples

Dual Optimization Problem

$$\max_{\alpha} \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j (x_i^T x_j)$$

Subject to:

$$\alpha_i \geq 0$$

$$\sum_{i=1}^{n} \alpha_i y_i = 0$$

(Soft margin: $0 \leq \alpha_i \leq C$)

- Instead of directly finding w,We find weights αi for each training sample.
- $x_i^T x_j$ represents similarity between two training samples. Dual form depends only on similarity.

After Solving:

$$w = \sum_{i=1}^{n} \alpha_i y_i x_i$$

Only points with:

$$\alpha_i > 0$$

become **Support Vectors**.

The Real Prediction Formula?

The correct decision function is:

$$f(x) = \sum_{i=1}^{n} \alpha_i y_i (x_i^T x) + b$$

Then predicted class is:

$$\text{sign}(f(x))$$

So:

- ✔ $f(x) \rightarrow$ score
- ✔ sign of $f(x) \rightarrow$ predicted class

$(x_i^T x)$ is similarity term

$$f(x) = \sum \alpha_i y_i (x_i^T x) + b$$

$$\text{Predicted class} = \text{sign}(f(x))$$

Let us assume we have 100 Observations

Training data:

$$(x_1, y_1), (x_2, y_2), \ldots, (x_{100}, y_{100})$$

So:

- 100 training vectors x_i
- 100 labels y_i

$$\max_{\alpha} \sum_{i=1}^{100} \alpha_i - \frac{1}{2} \sum_{i=1}^{100} \sum_{j=1}^{100} \alpha_i \alpha_j y_i y_j (x_i^T x_j)$$

Subject to:

$$0 \leq \alpha_i \leq C$$

$$\sum_{i=1}^{100} \alpha_i y_i = 0$$

C → **regularization parameter**

The only unknowns are:

$$\alpha_1, \alpha_2, \ldots, \alpha_{100}$$

So we are optimizing **100 variables**.

Each αi tells **how important that training point is in defining the decision boundary.**

What Does α Represent Here?

At the start:

All $\alpha_i = 0$

During optimization:

Some α_i increase from 0

At the end:

Most α_i return to 0

Some remain positive → support vectors

Example With 100 Observations

After solving, suppose:

Only 12 α_i are non-zero.

Then:

- 12 support vectors
- 88 irrelevant points

Why do we see $\alpha_i \alpha_j$?

- Because the dual objective considers **pairwise interactions between training samples. The** term means: $\alpha_i \alpha_j y_i y_j (x_i^T x_j)$ **interaction between sample i and sample j.**
- So the algorithm checks:
- how similar two points are?
- how their classes relate?

- **What do i and j mean?**
- They are **just indices**.
- Example if dataset has 3 samples:
- (x1,y1),(x2,y2),(x3,y3).
- Then the double summation evaluates:
- (i=1,j=1) (i=1,j=2)(i=1,j=3)
- (i=2,j=1)(i=2,j=2)(i=2,j=3)
- (i=3,j=1)(i=3,j=2)(i=3,j=3)

Example with 3 samples

The second term becomes:

$$\frac{1}{2}[\alpha_1\alpha_1 y_1 y_1 (x_1^T x_1) + \alpha_1\alpha_2 y_1 y_2 (x_1^T x_2) + \alpha_1\alpha_3 y_1 y_3 (x_1^T x_3) + \alpha_2\alpha_1 y_2 y_1 (x_2^T x_1) + \alpha_2\alpha_2 y_2 y_2 (x_2^T x_2) + \alpha_2\alpha_3 y_2 y_3 (x_2^T x_3) + \ldots]$$

So it compares **every training point with every other training point**.

- Dual SVM is computes **how every training sample interacts with every other sample**.
- That's why we see the double summation:
- ∑ i
- ∑ j

Suppose passenger-1 has features:

Age	Fare	Pclass
22	7.25	3

Then

$$x_1 = (22,\ 7.25,\ 3)$$

So x_1 is **the input vector of observation 1**.

For SVM we encode labels as:

$$y \in \{-1, +1\}$$

Symbol	Meaning
(x_1)	feature vector of sample 1
(y_1)	class label of sample 1
$(x_1^T x_1)$	similarity of sample with itself
$\alpha_1)$	weight assigned to sample 1

Where Does $y_i(w^T x_i + b)$ Appear?

That expression is used in **training**, not prediction.

During training, we use:

$$y_i(w^T x_i + b)$$

This tells whether a training point is:

- Correctly classified
- Inside margin
- Misclassified

If:

$$y_i(w^T x_i + b) \geq 1$$

→ Correct & outside margin

Problem: Titanic Survival

- We use only **Age** as feature.
- Label:
- Survived → +1
- Not Survived → −1
- Training Data: In table each one is training point(xi)

Passenger	Age	Survived (y)
P1	10	+1
P2	18	+1
P3	25	-1
P4	40	-1

Step 1: Assume SVM Finds Boundary at Age = 21

That means decision function:

$$f(x) = wx + b$$

Let's assume:

$$w = -0.5$$

$$b = 10.5$$

Check boundary:

$$-0.5x + 10.5 = 0$$

$$x = 21$$

Correct.

Step 2: Check Margin Condition

Compute:

$$y_i(wx_i + b)$$

Passenger 1 (Age 10, +1)

$$f(10) = -0.5(10) + 10.5 = 5.5$$

$$yf(x) = (+1)(5.5) = 5.5 > 1$$

Correct and far from boundary

→ α = 0

Passenger 2 (Age 18, +1)

$$f(18) = -9 + 10.5 = 1.5$$

$$yf(x) = 1.5 > 1$$

Close to margin

May become support vector.

Passenger 3 (Age 25, -1)

$$f(25) = -12.5 + 10.5 = -2$$

$$yf(x) = (-1)(-2) = 2 > 1$$

Correct and far

→ α = 0

Passenger 4 (Age 40, -1)

$$f(40) = -20 + 10.5 = -9.5$$

$$yf(x) = (-1)(-9.5) = 9.5 > 1$$

Far away

→ α = 0

Who Becomes Support Vector?

The closest opposite-class points:

Age 18 (+1)

Age 25 (−1)

These define margin.

So:

$$\alpha_{18} > 0$$

$$\alpha_{25} > 0$$

All others:

$$\alpha = 0$$

Step 3: Dual View

Now prediction for new passenger Age = 20:

Dual formula:

$$f(x) = \alpha_{18}(+1)(18 \cdot 20) + \alpha_{25}(-1)(25 \cdot 20) + b$$

Only 18 and 25 matter.

Others ignored.

Interpretation

- Instead of drawing line directly,
- **Dual says:**
- Compare new passenger with important passengers (18 and 25).
- If more similar to 18 → survived

  If more similar to 25 → not survived

Important Observations

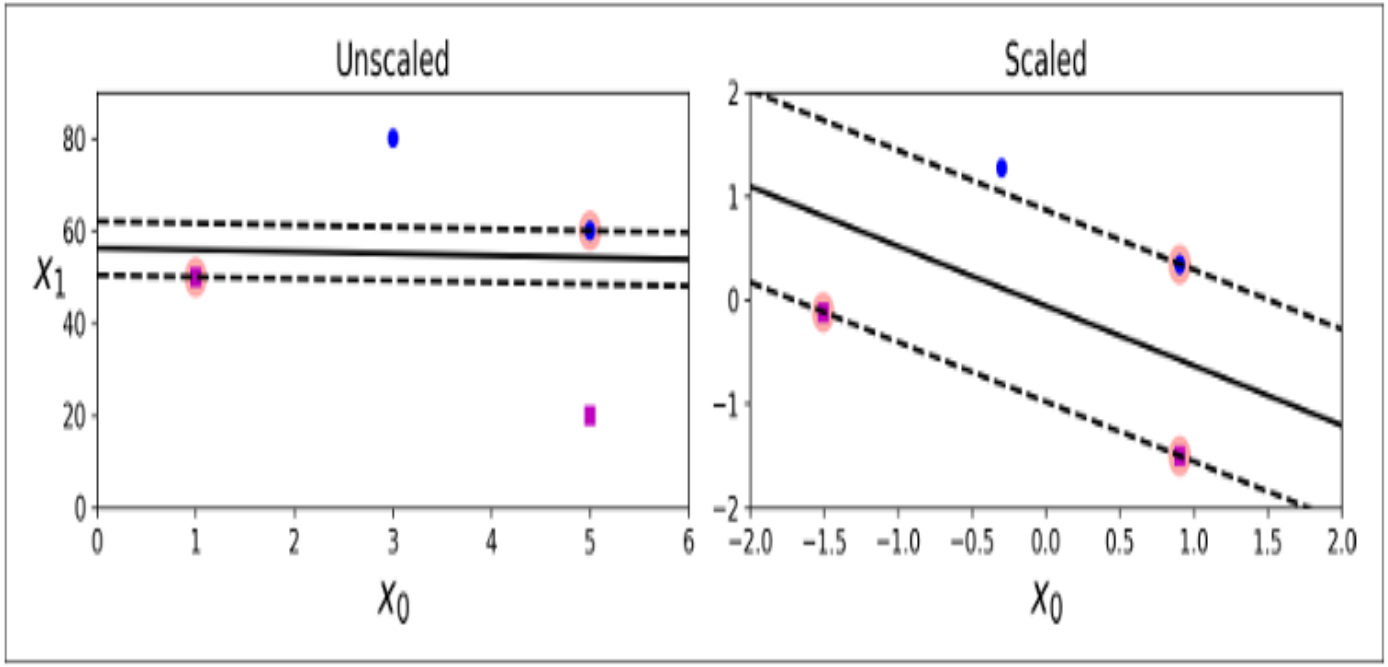
- α never negative
- Only borderline passengers get $\alpha > 0$
- Far passengers get $\alpha = 0$
- Misclassification depends on margin condition, not α sign

- Support vectors = passengers closest to decision boundary.
- They alone define survival rule.

Condition	Meaning
$\alpha_i = 0$	Not support vector, far from boundary
$0 < \alpha_i < C$	On margin
$\alpha_i = C$	Inside margin or misclassified
$\alpha_i < 0$	Impossible

SVM and Feature scaling

- SVMs are sensitive to the feature scales, as in Figure on the left plot, the vertical scale is much larger than the horizontal scale, so the widest possible street is close to horizontal.
- After feature scaling, the decision boundary looks much better (on the right plot).



Applications

- **Content:** SVM finds applications across various domains:
  - **Image Classification**: Identifying objects and patterns in images.
  - **Text Classification**: Categorizing documents based on their content.
  - **Bioinformatics**: Analyzing genetic data for disease diagnosis.
  - **Finance**: Predicting stock market trends and fraud detection.
  - **Medicine**: Diagnosing diseases based on medical data.

Key Points for SVM

1. **Outlier Handling:**

   If outliers are present, handle them appropriately as they can significantly affect the SVM model's performance.

2. **Overfitting and Under fitting:**

   Overfitting: Reduce C

   Under fitting: Increase C

3. **Hyper parameter Tuning:** Grid Search or Randomized Search

   - **C (Regularization parameter)**
   - **Kernel type**
   - **Gamma (for RBF kernel)**

4.High-Dimensional Data:

SVM performs well in high-dimensional spaces, making it a good choice for datasets with many features.

SVM works **very well when features are large (p >> n)**.

5.Kernel Selection: Choosing the right kernel is crucial for model performance:

- **Linear Kernel:** Suitable for linearly separable data.
- **Polynomial Kernel:** For data with polynomial relationships.
- **RBF Kernel (Radial Basis Function):** For non-linear data.
- **Sigmoid Kernel:** Rarely used; logistic regression is preferred for sigmoid-shaped data.

6. Observation vs. Variables:

SVM can handle high-dimensional data well.

However, training becomes expensive when the dataset size is extremely large.

7. Feature Scaling:

Always perform feature scaling (e.g., StandardScaler or MinMaxScaler) before training an SVM model. SVM is sensitive to the scale of the features.

8. Decision Surface:

SVM creates a decision boundary (or surface) that separates different classes. The margin is maximized between the closest points of different classes (support vectors).

9. Nonlinear Problems:

Linear SVM handles linearly separable data. Kernel SVM extends SVM to non-linear problems..

10. Feature Importance:

While SVM doesn't directly provide feature importance scores, techniques like Recursive Feature Elimination (RFE) can be used to identify important features.